

Blocco 6 – Aggregazioni e KPI

Problema, specifica, soluzione e implementazione

Materiali

- Durata: 90 minuti.
- Slide: `slides/blocks/block06_aggregazioni_kpi/block06_aggregazioni_kpi.pdf`
- Ambiente laboratorio: container Docker PostgreSQL `rdsq1-postgres`
- Schema PostgreSQL: `../..../sql/01_schema_seed_postgres.sql`
- Script soluzione: `solution.sql`
- Script unico schema + soluzioni: `../..../sql/00_schema_and_all_solutions_postgres.sql`

1. Problema in forma informale

Il team direzione vuole una pagina KPI: ricavo per mese e canale, ricavo per paese, AOV per segmento, quantità e ricavo per categoria prodotto, ticket medi.

2. Specifica corretta del problema

- input: viste e tabelle del dataset `training`;
- output: query aggregate con nome KPI, popolazione, granularità e ordinamento;
- vincolo: escludere ordini cancellati e rimborsati quando si parla di vendita valida;
- vincolo: usare `HAVING` solo per filtri su gruppi aggregati.

3. Definizione della soluzione

1. creare prima query semplici per ogni KPI;
2. aggiungere `GROUP BY` solo dopo aver definito la riga finale;
3. usare `FILTER` per conteggi condizionali di stati;
4. validare con conteggi di ordini e somme totali.

4. Implementazione completa o artefatto atteso

La soluzione seguente è caricabile nel container PostgreSQL Docker dopo lo schema di laboratorio.

```
SET search_path TO training;

SELECT date_trunc('month', order_date)::date AS month,
       channel,
       round(sum(gross_revenue), 2) AS revenue
FROM order_revenue
WHERE status NOT IN ('cancelled', 'refunded')
GROUP BY month, channel
```

```

ORDER BY month, channel;

SELECT c.country,
       count(DISTINCT r.order_id) AS orders,
       round(sum(r.gross_revenue), 2) AS revenue
FROM order_revenue r
JOIN customers c ON c.customer_id = r.customer_id
WHERE r.status NOT IN ('cancelled', 'refunded')
GROUP BY c.country
ORDER BY revenue DESC;

SELECT c.segment,
       count(*) AS valid_orders,
       round(sum(r.gross_revenue), 2) AS revenue,
       round(avg(r.gross_revenue), 2) AS avg_order_value
FROM order_revenue r
JOIN customers c ON c.customer_id = r.customer_id
WHERE r.status NOT IN ('cancelled', 'refunded')
GROUP BY c.segment
HAVING count(*) >= 2
ORDER BY revenue DESC;

SELECT channel,
       count(*) FILTER (WHERE status = 'completed') AS completed_orders,
       count(*) FILTER (WHERE status = 'shipped') AS shipped_orders,
       count(*) FILTER (WHERE status = 'pending') AS pending_orders
FROM orders
GROUP BY channel
ORDER BY channel;

```

5. Suggerimenti

- prima di lavorare sulla soluzione, riscrivere la domanda con parole proprie;
- esplicitare input, output, vincoli e assunzioni;
- controllare la granularità del risultato atteso;
- preparare almeno un caso limite da discutere in aula.

6. Criteri di verifica

- il KPI è definito in parole prima del codice;
- la granularità del risultato è leggibile dalle colonne in `GROUP BY`;
- denominatore e stati inclusi sono espliciti;
- i controlli confermano che il join non ha duplicato i fatti.

7. Checkpoint

- una metrica senza denominatore dichiarato è ambigua;
- `GROUP BY` cambia la granularità: va letto come una decisione di business;
- un risultato aggregato corretto si difende con controlli intermedi.